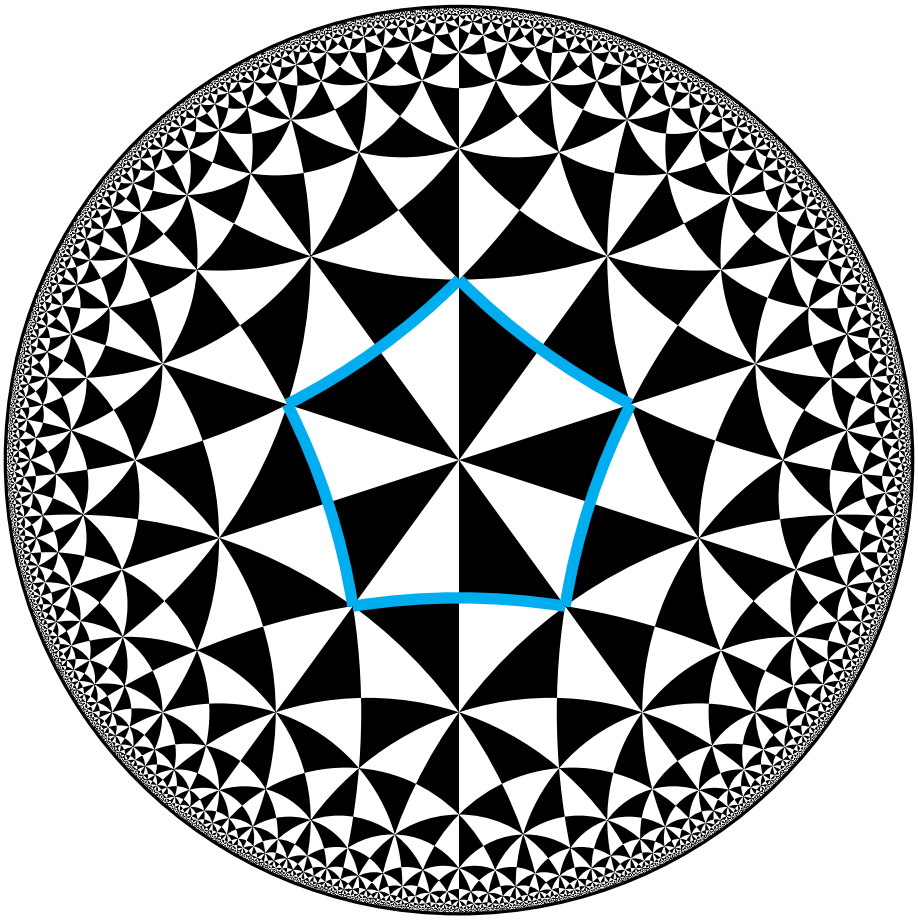
Package luahyperbolic

Damien Mégy



The latest version of this documentation is available at: github.com/dmegy/luahyperbolic.

Contents

1 Introduction **2**
- 1.1 What you can do with this package . . . 2
- 1.2 What this package cannot do for you . . 2
- 1.3 Efficiency 2
- 1.4 How to load the package 2
- 1.5 How to customize the package 3
- 1.6 Environment hyperbolicdisk 3
  - 1.6.1 Exporting the pictures 3

2 Drawing and labeling elements **4**
- 2.1 Points 4
- 2.2 Geodesic segments, lines and rays 4
- 2.3 Circles, semicircles, arcs 7
- 2.4 Horocycles and hypercycles 9
- 2.5 Printing custom strings into the TikZ picture 9

3 Defining points **10**
- 3.1 Random points 10
- 3.2 Midpoint, interpolate, barycenter 11
- 3.3 Intersections 11
- 3.4 Reflections and projections 12
- 3.5 Automorphisms (direct isometries) . . . 13
- 3.6 Orbits under automorphisms 15
- 3.7 Triangle geometry (CAUTION) 16

4 Tilings **19**
- 4.1 Source code of first picture 19
- 4.2 More images just for fun 20

5 Overview of the lua libraries **21**
- 5.1 complex.lua 21
- 5.2 luahyperbolic-tikz (requires and copies luahyperbolic-core) 21

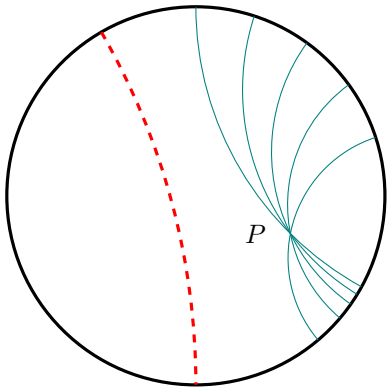
1 Introduction

1.1 What you can do with this package

1. Draw hyperbolic geometry pictures, high precision (pdf output).
2. Use an easy syntax. At least, easier than with LaTeXmacros and bloody pgf.
3. Export generated pictures to external tikz files if you want.
4. Read the source code and learn hyperbolic geometry.
5. Compile **without** installing python packages or lua packages etc.
6. Compile **without** "shell-escape" mode.
7. Compile everything in a single pass. No (Leeloo Dallas) multipass. Want to illustrate Euclid's fifth postulate ? Do it right there in your tex file without having to include external files :

Euclid's 5th Postulate

```
\begin{luacode*}
hyper.tikzBegin("scale=2.5")
local P = complex(0.5,-0.2)
local A = complex.exp_i(math.pi/10)
for k=1,5 do
  hyper.drawLine(P, A^k, "teal")
end
hyper.labelPoint(P, "$P$", "left=.2cm")
local style = "very thick, dashed, red"
hyper.drawLine(complex.J,-complex.I,style)
hyper.tikzEnd()
\end{luacode*}
```



1.2 What this package cannot do for you

1. Really intensive computations, millions of geodesics etc. Source code is not optimized to keep maximum readability. All computations involve automorphisms. See other remarks below.
2. Coffee

1.3 Efficiency

For heavy computations, please use a ~~Python~~ C library. Nevertheless, this library is still reasonably efficient and usable:

- The first tiling illustration of this documentation (hyperbolic 2-4-5 tiling with 2361 geodesics) compiled in 3.21 seconds on a 2020 intel macbook air laptop.
- This entire documentation including all pictures compiles in approx. 15 seconds on the same machine, the majority of the time weing spent on the tilings. The package is suitable for producing lecture notes in hyperbolic geometry, which was the reason for all of this. For an entire book with dozens of pictures, you may need to externalize some of the pictures.
- Tests show that the bottleneck is TikZ, not Lua. In a future version, some future maintainer may add other fontends to the library, including drawing directly to the pdf with pdf primitives wich are several orders of magnitude faster than tikz.

1.4 How to load the package

Local install

Simply put the `luahyperbolic.sty` file in your working directory and add the following line to your tex preamble :

```
\usepackage{luahyperbolic}
```

The file `luahyperbolic.sty` **contains everything needed**, including the necessary library for complex numbers. It can be sent to colleagues by email etc.

It will declare two lua globals : `complex` and `hyper`, and define a `hyperbolicdisk` environment for convenience.

If the sty file is not in the same directory as your tex file, you can write `\usepackage{../luahyperbolic}`

CTAN

Not available yet.

1.5 How to customize the package

The `luahyperbolic.sty` is generated from three lua modules : `complex.lua`, `luahyperbolic-core.lua` and `luahyperbolic-tikz.lua`. These files are in the `lua/` subdirectory of the package repo.

To build the `luahyperbolic.sty` from lua source files, execute `lua build.lua` in the root directory of the package. This will backup the old sty file to the `backup/` directory, then buid and replace the existing sty file.

1.6 Environment hyperbolicdisk

The package provides the latex environment `\begin{hyperbolicdisk} ... \end{hyperbolicdisk}` which is defined as follows:

```
\newenvironment{hyperbolicdisk}[1][scale=3]{%
  \begin{tikzpicture}[#1]
    \draw[very thick] (0,0) circle (1);
    \clip (0,0) circle (1);
}{%
  \end{tikzpicture}
}
```

In this documentation, we prefer to use the lua functions `hyper.tikzBegin(options)` and `hyper.tikzEnd(export_file)` that enable export functionalities, see below.

1.6.1 Exporting the pictures

Simply add a filename to the closing `hyper.tikzEnd()` instruction, as below:

```
hyper.tikzEnd("myfile.tikz")
```

This will save the current picture (including `\begin{tikzpicture}[options]` and `\end{tikzpicture}`).

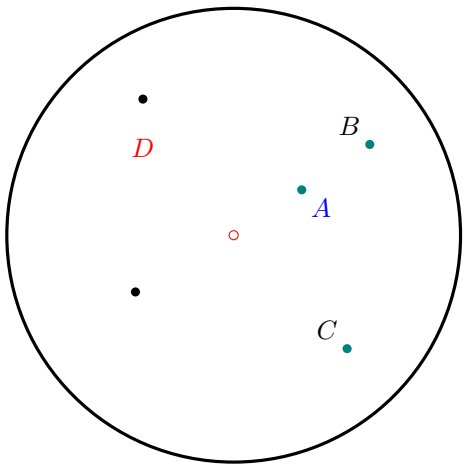
It is also possible to export the current picture at any point with `hyper.tikzExport(optional-filename)`. If no filename is provided, the package will name it `hyper_picture_N` with `N` the picture counter. Luahyperbolic keeps count of how many pictures have been produced in the document. This allows exporting images in loops, or drawing several steps of the same picture.

2 Drawing and labeling elements

2.1 Points

Draw and label points

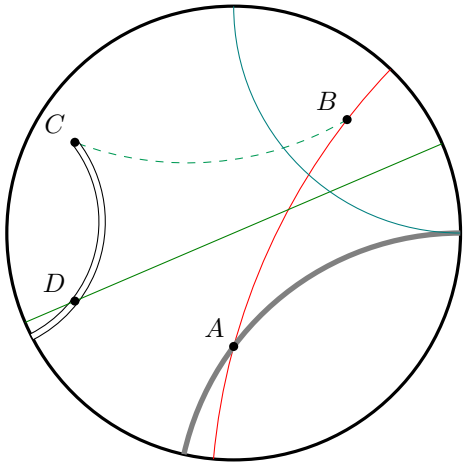
```
\begin{luacode*}
local O = 0
local A = complex(0.3,0.2)
local B = 2*A
local C = (1-complex.I)/2
local D = B*complex.I
local E = complex.exp_i(-5*math.pi/6)/2
hyper.tikzBegin()
hyper.drawPoint(O,"white, draw=red")
hyper.drawPoints(A, B, C, "teal")
hyper.drawPoints(D, E)
hyper.labelPoint(A,"$A$", "below right, blue")
hyper.labelPoints(B,"$B$", C, "$C$")
hyper.labelPoint(D,"$D$","below=.4cm, red")
hyper.tikzEnd()
\end{luacode*}
```



2.2 Geodesic segments, lines and rays

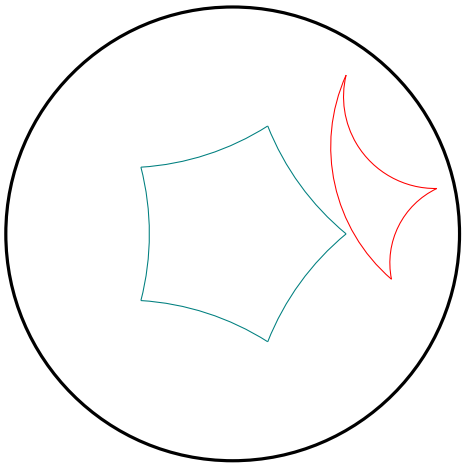
Segments lines and rays (through two points)

```
\begin{luacode*}
local A = -complex.I/2
local B = (1+complex.I)/2
local C = complex(-0.7,0.4)
local D = complex(-0.7,-0.3)
hyper.tikzBegin()
hyper.drawLine(A, B, "red")
hyper.drawSegment(B,C,"dashed, ForestGreen")
hyper.drawRay(C,D,"double,double distance=2pt")
hyper.drawLine(D,-D, "green!50!black")
hyper.drawLine(A, 1, "gray, line width=2pt")
hyper.drawLine(1, complex.I, "teal")
hyper.drawPoints(A, B, C, D)
hyper.labelPoints(A, "$A$", B, "$B$", C, "$C$",
    ↪ D, "$D$")
hyper.tikzEnd()
\end{luacode*}
```



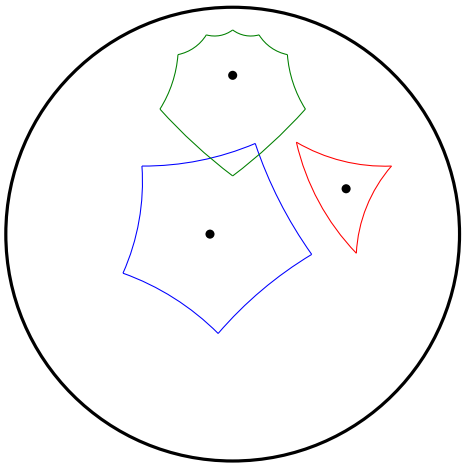
Triangles and polygons

```
\begin{luacode*}
hyper.tikzBegin()
hyper.drawTriangle(
  complex(.9,.2),
  complex(.5,.7),
  complex(.7,-.2), "red")
local points={}
for k=1,5 do
  table.insert(points, complex.polar(.5,k*2*
      ↪ math.pi/5))
end
hyper.drawPolygonFromTable(points,"teal")
hyper.tikzEnd()
\end{luacode*}
```



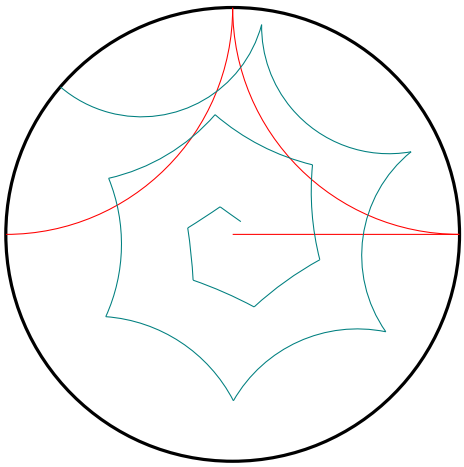
Regular polygons

```
\begin{luacode*}
local O1 = complex(.5,.2)
local A1 = complex(.7,.3)
local O2 = complex(-0.1,0)
local A2 = complex(-.4,.3)
local O3 = complex(0,.7)
local A3 = complex(0,.9)
hyper.tikzBegin()
hyper.drawRegularPolygon(O1, A1, 3, "red")
hyper.drawRegularPolygon(O2, A2, 5, "blue")
hyper.drawRegularPolygon(O3, A3, 8, "green!50!
    ↪ black")
hyper.drawPoints(O1,O2,O3)
hyper.tikzEnd()
\end{luacode*}
```



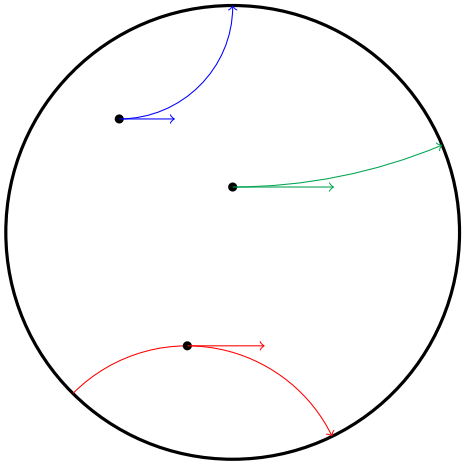
Polygonal lines

```
\begin{luacode*}
hyper.tikzBegin()
hyper.drawPolyline(0, 1, complex.I, -1, "red")
local points={}
for k=1, 15 do
  table.insert(points, complex.polar(k/15, k))
end
hyper.drawPolylineFromTable(points,"teal")
hyper.tikzEnd()
\end{luacode*}
```



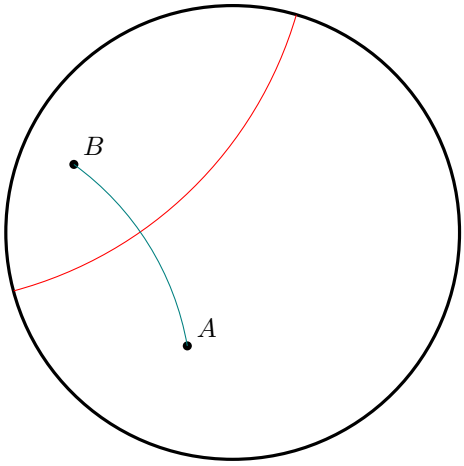
Lines and rays (point and tangent vector)

```
\begin{luacode*}
local A = complex(-0.2,-0.5)
local B = complex(-0.5,0.5)
local C = complex(0,0.2)
local v =  complex(1,0)
hyper.tikzBegin()
hyper.drawLineFromVector(A, v, "->, red")
hyper.drawRayFromVector(B, v, "->, blue")
hyper.drawRayFromVector(C, v, "->, Green")
hyper.drawPoints(A, B,C)
hyper.drawTangentVector(A,v,"red")
hyper.drawTangentVector(B,v,"blue")
hyper.drawTangentVector(C,v,"Green")
hyper.tikzEnd()
\end{luacode*}
```



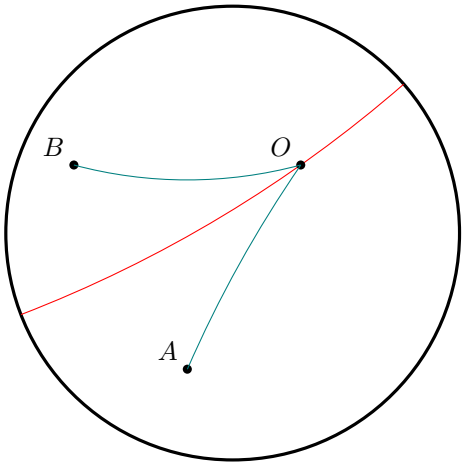
Perpendicular bisector

```
\begin{luacode*}
local A = complex(-0.2,-0.5)
local B = complex(-0.7,0.3)
hyper.tikzBegin()
hyper.drawPerpendicularBisector(A,B,"red")
hyper.drawPoints(A,B)
hyper.drawSegment(A, B, "teal")
hyper.labelPoints(A,"$A$", B, "$B$", "above
    ↪ right")
hyper.tikzEnd()
\end{luacode*}
```



Angle bisector

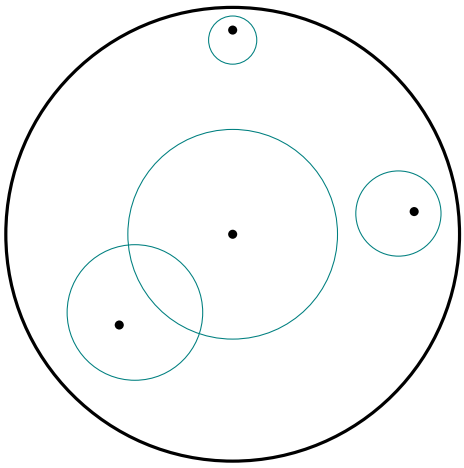
```
\begin{luacode*}
local A = complex(-0.2,-0.6)
local O = complex(0.3,0.3)
local B = complex(-0.7,0.3)
hyper.tikzBegin()
hyper.drawAngleBisector(A,O,B,"red")
hyper.drawPoints(A,O,B)
hyper.drawPolyline(A,O,B, "teal")
hyper.labelPoints(A, "$A$", O, "$O$", B, "$B$")
hyper.tikzEnd()
\end{luacode*}
```



2.3 Circles, semicircles, arcs

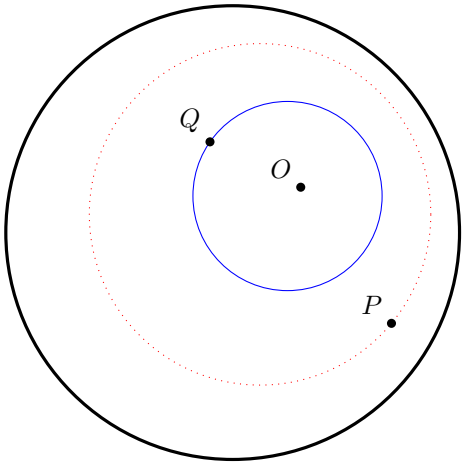
Center and radius

```
\begin{luacode*}
local points = {
  complex(0,0),
  complex(-0.5,-0.4),
  complex(0.8,0.1),
  complex(0,0.9)
}
hyper.tikzBegin()
for _,point in ipairs(points) do
  hyper.drawCircle(point,1,"teal")
  hyper.drawPoint(point)
end
-- NOTE : all circles have same radius=1 !
hyper.tikzEnd()
\end{luacode*}
```



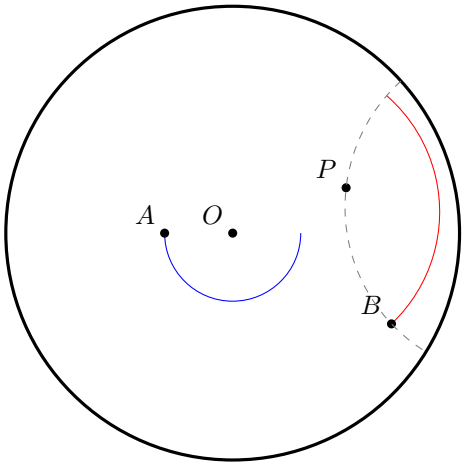
Circle through point

```
\begin{luacode*}
local O = complex(0.3,0.2)
local P = complex(0.7,-0.4)
local Q = complex(-0.1,0.4)
hyper.tikzBegin()
hyper.drawCircleThrough(O, P, "dotted, red")
-- or draw by hand :
hyper.drawCircle(O, hyper.distance(O,Q), "blue"
    ↪ )
hyper.drawPoints(O, P, Q)
hyper.labelPoints(O, "$O$", P, "$P$", Q, "$Q$")
hyper.tikzEnd()
\end{luacode*}
```



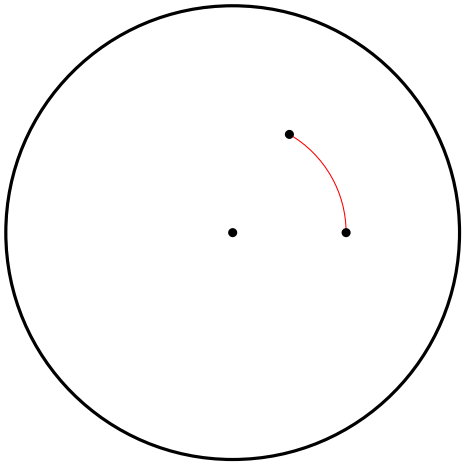
Semicircles

```
\begin{luacode*}
local O = complex(0,0)
local A = complex(-0.3,0)
local P = complex(0.5,0.2)
local B = complex(0.7,-0.4)
hyper.tikzBegin()
hyper.drawSemicircle(O, A, "blue")
hyper.drawSemicircle(P, B, "red")
hyper.drawPoints(O, A, P, B)
hyper.labelPoints(O, "$O$", A, "$A$", P, "$P$",
    ↪  B, "$B$")
hyper.drawLine(P,B, "gray, dashed")
hyper.tikzEnd()
\end{luacode*}
```



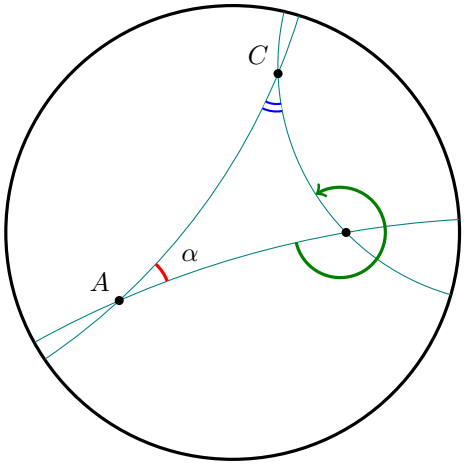
Arc (center and endpoints)

```
\begin{luacode*}
local O = complex(0,0)
local A = complex(0.5,0)
local B = complex.polar(0.5,math.pi/3)
hyper.tikzBegin()
hyper.drawArc(O,A,B, "red")
hyper.drawPoints(O,A,B)
hyper.tikzEnd()
\end{luacode*}
```



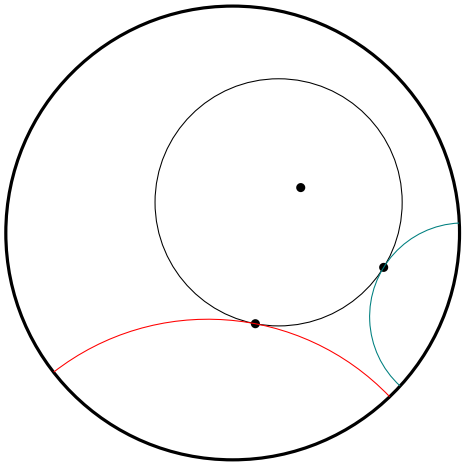
Angles

```
\begin{luacode*}
local A = complex(-.5,-.3)
local B = complex(0.5,0)
local C = complex(0.2,.7)
hyper.tikzBegin()
hyper.drawLines(A, B, B, C, C, A, "teal")
hyper.drawAngle(B, A, C, "red, very thick")
hyper.drawAngle(A, C, B, "thick, double, double
    ↪  distance=2pt, blue")
hyper.drawAngle(A, B, C, "very thick, ->, green
    ↪ !50!black")
hyper.drawPoints(A, B, C)
hyper.labelPoints(A, "$A$", C, "$C$")
hyper.labelPoint(N, "$B$", "above right")
-- poor man's labelAngle:
hyper.labelPoint(A,"$\\alpha$", "above=.6cm,
    ↪ right=.7cm")
hyper.tikzEnd()
\end{luacode*}
```



Tangent at point

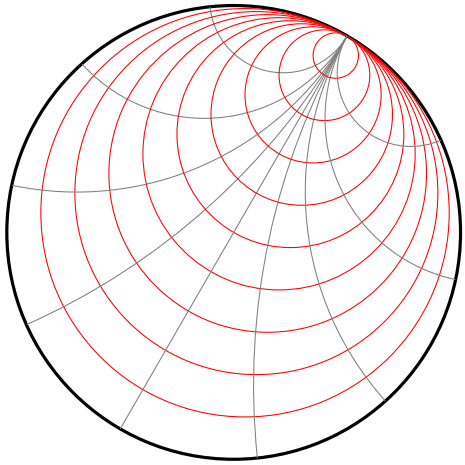
```
\begin{luacode*}
local O = complex(0.3,0.2)
local P = complex(0.1,-0.4)
local Q = hyper.rotation(O,math.pi/3)(P)
hyper.tikzBegin()
hyper.drawCircleThrough(O, P)
hyper.drawPoints(O, P, Q)
hyper.drawTangentAt(O,P,"red")
hyper.drawTangentAt(O,Q,"teal")
hyper.tikzEnd()
\end{luacode*}
```



2.4 Horocycles and hypercycles

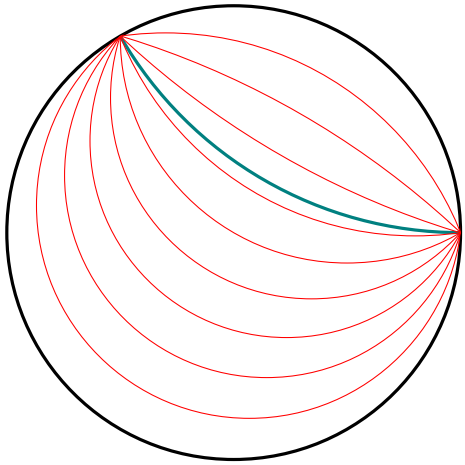
Horocycles

```
\begin{luacode*}
local theta=math.pi/3
local idealPoint = complex.exp_i(theta)
local N = 10
hyper.tikzBegin()
for k=1,N-1 do
  local point = idealPoint * (2*k/N-1)
  hyper.drawHorocycle(idealPoint,point,"red")
  -- and an orthogonal geodesic :
  local endPoint = complex.exp_i(theta+2*k*math
      ↪ .pi/N)
  hyper.drawLine(idealPoint,endPoint,"gray")
end
hyper.tikzEnd()
\end{luacode*}
```



Hypercycles

```
\begin{luacode*}
hyper.tikzBegin()
local A = 1
local B = complex.J
hyper.drawLine(A,B,"very thick, teal")
local N = 10
for k=1,N-1 do
  local P = (2*k/N-1)*(-B^2)
  hyper.drawHypercycleThrough(P, A, B, "red")
end
hyper.tikzEnd()
\end{luacode*}
```



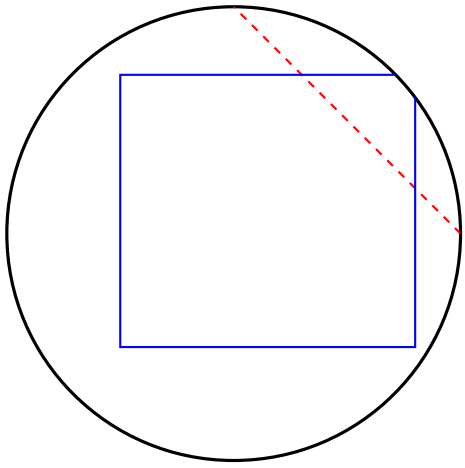
2.5 Printing custom strings into the TikZ picture

In case you want to insert custom TikZ instructions into the TikZ picture, the library provides a `tikzPintf()` function, which is essentially `tex.print` composed with `string.format()`, but also saves the printed string to the tikz buffer.

For example, you can write

Printing custom strings to TikZ

```
\begin{luacode*}
local A = complex.ONE
local B = complex.I
local C = (-A-B)/2
hyper.tikzBegin()
hyper.tikzPrintf(
  "\\draw[%s] (%f,%f) -- (%f,%f) ;",
  "red, thick, dashed", A.re, A.im, B.re, B.im
)
hyper.tikzPrintf(
  "\\draw[%s] (%f,%f) rectangle (%f,%f) ;",
  "blue, thick", C.re, C.im, 0.8,0.7
)
hyper.tikzEnd()
\end{luacode*}
```



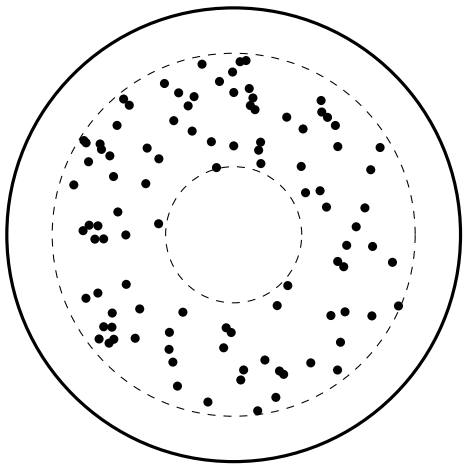
The package doesn't expose euclidean drawing functions. Users should write TikZ directly in the picture as above with `tikzPrintf`, or use some other library, for example the **amazing** library `tkz-euclide`.

3 Defining points

3.1 Random points

Random points with uniform density

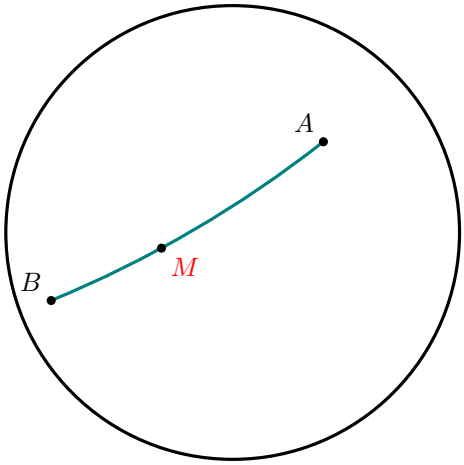
```
\begin{luacode*}
local rMin, rMax = .3, .8
hyper.tikzBegin()
for k=1,100 do
  local M = hyper.randomPoint(rMin,rMax)
  hyper.drawPoint(M)
end
hyper.tikzPrintf("\\draw[dashed] (0,0) circle
    ↪ (%s) ;", rMin)
hyper.tikzPrintf("\\draw[dashed] (0,0) circle
    ↪ (%s) ;", rMax)
hyper.tikzEnd()
\end{luacode*}
```



3.2 Midpoint, interpolate, barycenter

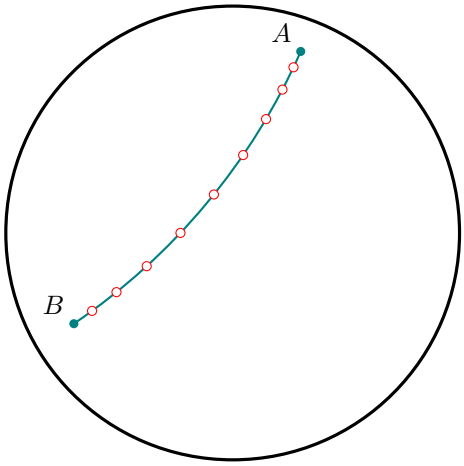
Midpoint

```
\begin{luacode*}
local A = complex(0.4,0.4)
local B = complex(-0.8,-0.3)
local M = hyper.midpoint(A,B)
hyper.tikzBegin()
hyper.drawSegment(A,B, "very thick, teal")
hyper.drawPoints(A,B,M)
hyper.labelPoints(A, "$A$", B, "$B$")
hyper.labelPoint(M, "$M$", "below right, red")
hyper.tikzEnd()
\end{luacode*}
```



Interpolate and barycenter

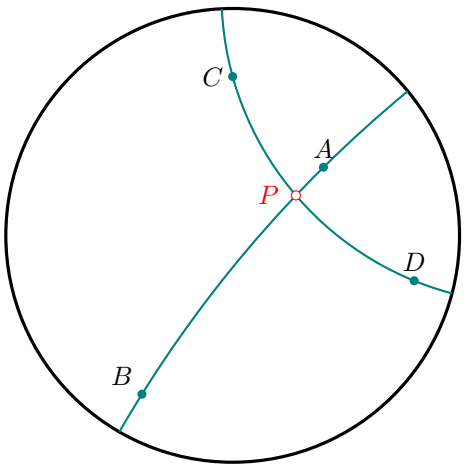
```
\begin{luacode*}
local A = complex(0.3,0.8)
local B = complex(-0.7,-0.4)
local N = 10
hyper.tikzBegin()
hyper.drawSegment(A,B, "thick, teal")
hyper.drawPoints(A, B, "teal")
hyper.labelPoints(A, "$A$", B, "$B$")
for k=1,N-1 do
  local M = hyper.interpolate(A, B, k/N)
  hyper.drawPoint(M, "white, draw=red")
end
hyper.tikzEnd()
\end{luacode*}
```



3.3 Intersections

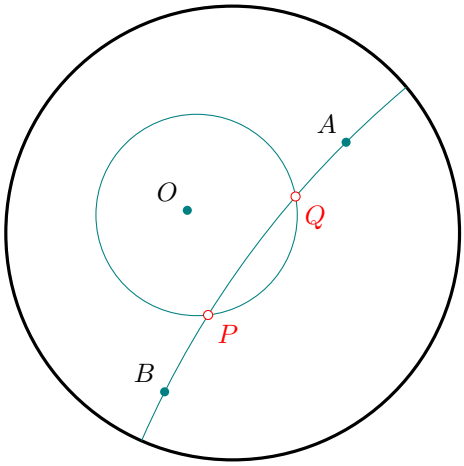
Line-Line (LL) intersection

```
\begin{luacode*}
local A = complex(0.4,0.3)
local B = complex(-0.4,-0.7)
local C = complex(0,0.7)
local D = complex(0.8,-0.2)
local P = hyper.interLL(A, B, C, D)
hyper.tikzBegin()
hyper.drawLines(A, B, C, D, "thick, teal")
hyper.drawPoints(A, B, C, D,"thick, teal")
hyper.drawPoint(P,"white, draw=red")
hyper.labelPoint(A, "$A$", "above")
hyper.labelPoint(B, "$B$")
hyper.labelPoint(C, "$C$", "left")
hyper.labelPoint(D, "$D$", "above")
hyper.labelPoint(P, "$P$", "left=.1cm, red")
hyper.tikzEnd()
\end{luacode*}
```



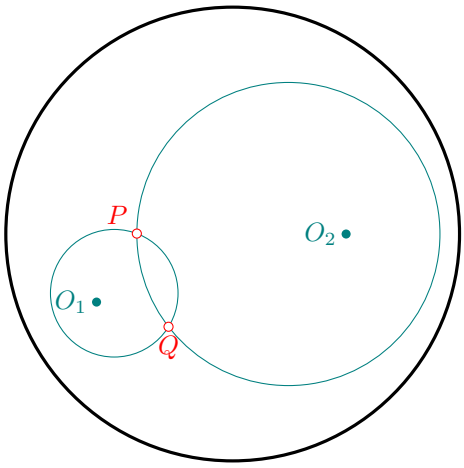
Line-circle (LC) intersection

```
\begin{luacode*}
local A = complex(0.5,0.4)
local B = complex(-0.3,-0.7)
local O = complex(-0.2,0.1)
local radius = 1
local P, Q = hyper.interLC(A, B, O, radius)
hyper.tikzBegin()
hyper.drawLine(A, B, "teal")
hyper.drawCircle(O,radius, "teal")
hyper.drawPoints(A, B, O, "teal")
hyper.drawPoints(P, Q, "white, draw=red")
hyper.labelPoints(A, "$A$", B, "$B$", O, "$O$")
hyper.labelPoints(P, "$P$", Q, "$Q$", "red,
    ↪ below right")
hyper.tikzEnd()
\end{luacode*}
```



Circle-circle (CC) intersection

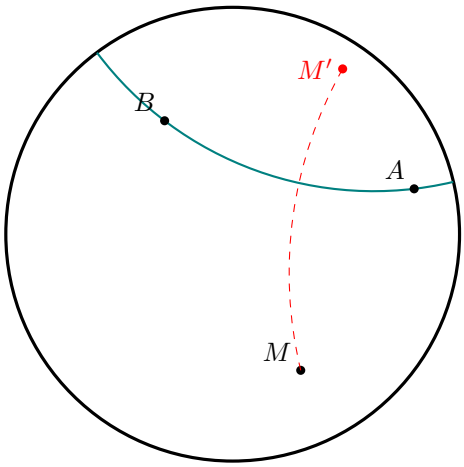
```
\begin{luacode*}
local O1 = complex(-0.6,-.3)
local O2 = complex(0.5,0)
local r1, r2 = 1, 2
local P, Q = hyper.interCC(O1, r1, O2, r2)
hyper.tikzBegin()
hyper.drawCircle(O1,r1, "teal")
hyper.drawCircle(O2,r2, "teal")
hyper.drawPoints(P, Q, "white, draw=red")
hyper.drawPoints(O1, O2, "teal")
hyper.labelPoints(O1, "$O_1$", O2, "$O_2$", "
    ↪ left, teal")
hyper.labelPoints(P, "$P$", "above left, red")
hyper.labelPoint(Q, "$Q$", "red, below")
hyper.tikzEnd()
\end{luacode*}
```



3.4 Reflections and projections

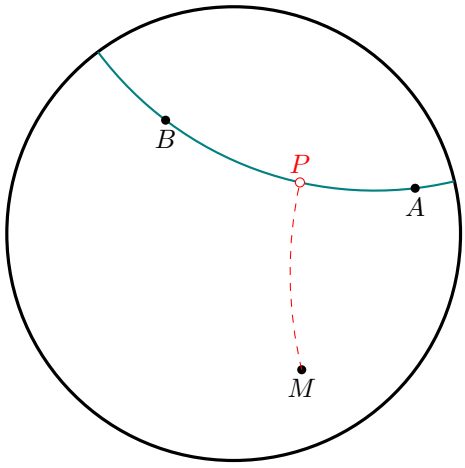
Reflection across a geodesic line

```
\begin{luacode*}
local A = complex(0.8,0.2)
local B = complex(-0.3,0.5)
local M = complex(0.3,-0.6)
local Mp = (hyper.reflection(A,B))(M)
hyper.tikzBegin()
hyper.drawLine(A, B, "thick, teal")
hyper.drawPoints(A, B, M)
hyper.drawPoint(Mp,"red")
hyper.drawSegment(M,Mp,"dashed,red")
hyper.labelPoint(Mp,"$M'$", "left, red")
hyper.labelPoints(A, "$A$", B, "$B$", M, "$M$")
hyper.tikzEnd()
\end{luacode*}
```



Projection onto a geodesic line

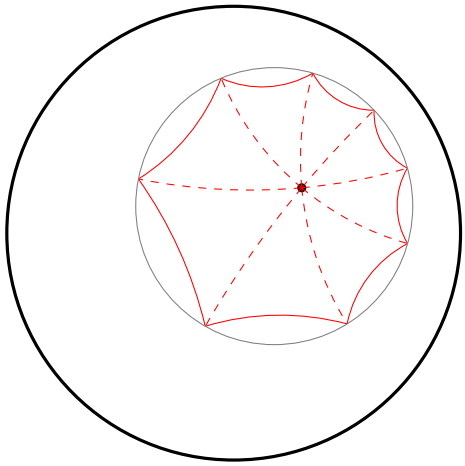
```
\begin{luacode*}
local A = complex(0.8,0.2)
local B = complex(-0.3,0.5)
local M = complex(0.3,-0.6)
local P = (hyper.projection(A,B))(M)
hyper.tikzBegin()
hyper.drawLine(A, B, "thick, teal")
hyper.drawPoints(A, B, M)
hyper.drawPoint(P,"white, draw=red")
hyper.drawSegment(M,P,"dashed,red")
hyper.labelPoint(P,"$P$", "above, red")
hyper.labelPoints(A, "$A$", B, "$B$", M, "$M$",
    ↪  "below")
hyper.tikzEnd()
\end{luacode*}
```



3.5 Automorphisms (direct isometries)

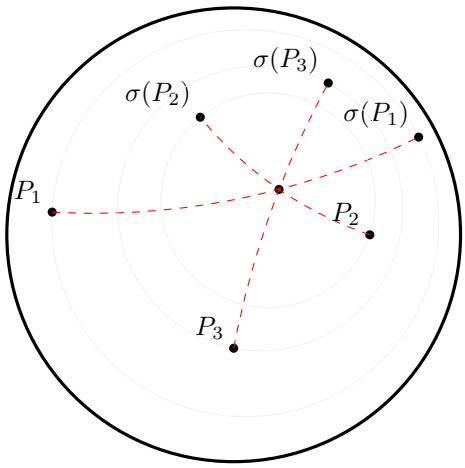
(Elliptic) rotation around a point (regular octogon)

```
\begin{luacode*}
local O = complex(0.3,0.2)
local M = complex(0.5,-0.4)
hyper.tikzBegin()
hyper.drawPoints(O)
for k=1,8 do
  local P = (hyper.rotation(O,2*math.pi/8))(M)
  hyper.drawSegment(M, P, "red")
  hyper.drawSegment(O,M, "dashed,red")
  M = P
end
hyper.drawCircle(O,hyper.distance(O,M),"thin,
    ↪ gray")
hyper.tikzEnd()
\end{luacode*}
```



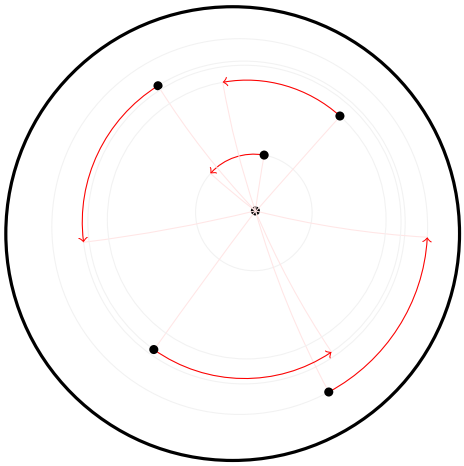
Symmetry around a point (rotation by 180°)

```
\begin{luacode*}
local O = complex(0.2,0.2)
local s = hyper.symmetry(O)
local points={complex(-.8,.1), complex(.6,0),
    ↪ complex(0,-.5)}
hyper.tikzBegin()
hyper.drawPoint(O)
for k, P in ipairs(points) do
  hyper.drawCircleThrough(O, P, "gray!10")
  local sP = s(P)
  hyper.drawPoints(P,sP)
  hyper.drawSegment(P,sP, "red, dashed")
  hyper.labelPoint(P, "$P_{".. k .."}$")
  hyper.labelPoint(sP, "$\\sigma(P_{".. k ..."})
      ↪ $")
end
hyper.tikzEnd()
\end{luacode*}
```



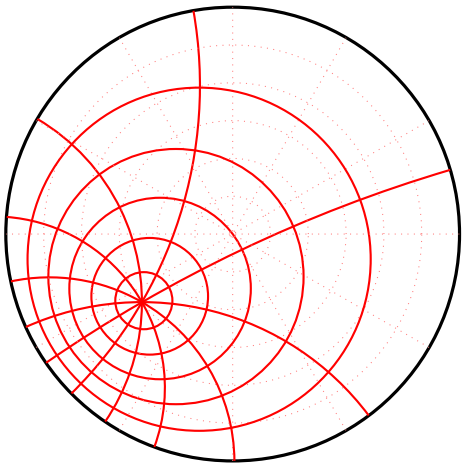
Rotation by 60°

```
\begin{luacode*}
local O = complex(0.1,0.1)
local r = hyper.rotation(O,math.pi/3)
hyper.tikzBegin()
hyper.drawPoint(O)
for k=1,5 do
  local P = hyper.randomPoint(0.3,0.9)
  local rP = r(P)
  hyper.drawCircleThrough(O,P, "gray!10")
  hyper.drawArc(O,P,rP,"->, red")
  hyper.drawPolyline(P,O,rP, "red!10")
  hyper.drawPoint(P)
end
hyper.tikzEnd()
\end{luacode*}
```



Hyperbolic automorphims

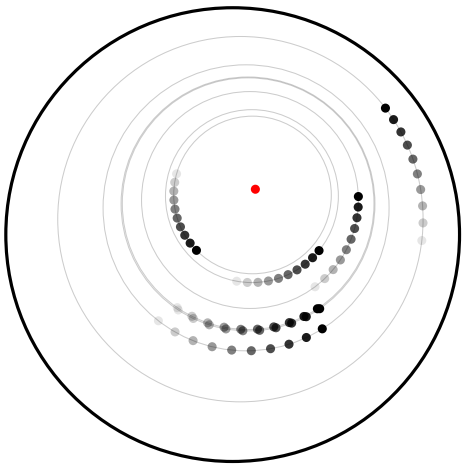
```
\begin{luacode*}
local A = complex(0.4,0.3)
local phi = hyper.automorphism(A,0)
local N=6
hyper.tikzBegin()
style1 = "dotted, red!50"
style2 =  "thick, red"
for k=1,N do
  local P = complex.polar(k/N, k*math.pi/N)
  local Q = phi(P)
  hyper.drawLine(0,P,style1)
  hyper.drawLine(-A,Q,style2)
  if k < N then
    hyper.drawCircleThrough(0,P, style1)
    hyper.drawCircleThrough(-A,Q,style2)
  end
end
hyper.tikzEnd()
\end{luacode*}
```



3.6 Orbits under automorphisms

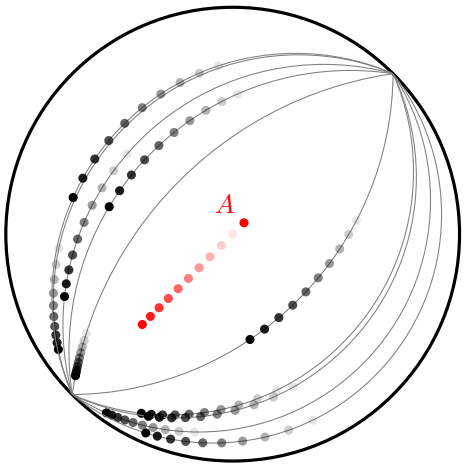
Orbit under elliptic automorphism (rotation)

```
\begin{luacode*}
local nbPoints = 7
local nbIterations = 10
local O = complex(0.1,0.2)
local rot = hyper.rotation(O,math.pi/30) --
    ↪ angle in radians
hyper.tikzBegin()
hyper.drawPoint(O,"red")
for k=1,nbPoints do
  local P = hyper.randomPoint(0,.9)
  hyper.drawCircleThrough(O,P,"draw opacity=0.2
      ↪ ")
  hyper.drawPointOrbit(P, rot, nbIterations)
end
hyper.tikzEnd()
\end{luacode*}
```



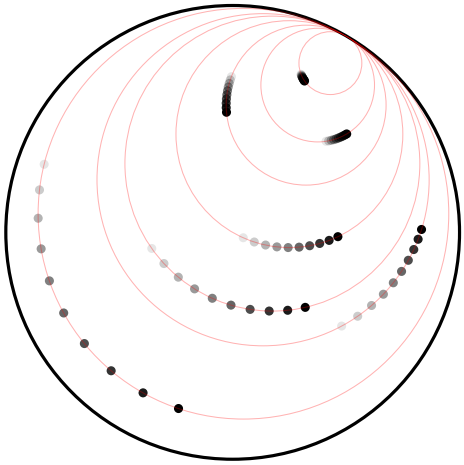
Orbit under hyperbolic automorphism

```
\begin{luacode*}
local nbPoints = 10
local nbIterations = 10
local A = complex(0.05,0.05)
local phi = hyper.automorphism(A,0)
local X, Y = hyper.endpoints(A,0)
hyper.tikzBegin()
for k=1,nbPoints do
  local P = hyper.randomPoint(0,.9)
  hyper.drawHypercycleThrough(P,X,Y,"gray")
  hyper.drawPointOrbit(P, phi, nbIterations)
end
hyper.drawPoint(A,"red")
hyper.labelPoint(A,"$A$", "above left, red")
hyper.drawPointOrbit(A, phi, nbIterations, "red
    ↪ ")
hyper.tikzEnd()
\end{luacode*}
```



Orbit under parabolic automorphism

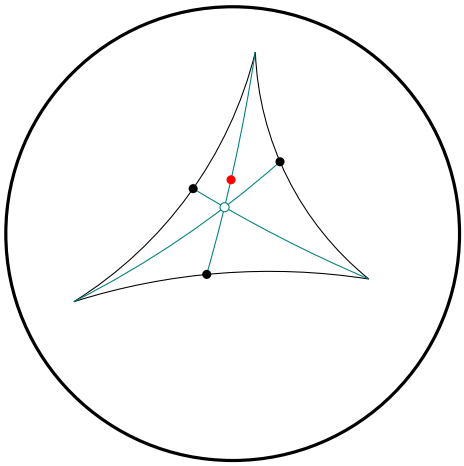
```
\begin{luacode*}
local nbIterations = 10
local limit = complex.exp_i(math.pi/3) -- on
    ↪ circle
local phi = hyper.parabolic(limit,math.pi/30)
local points={
  complex(0,0), complex(0,.7),
  complex(.4,.4), complex(.3,.7),
  complex(-.8,.4), complex(-.4,0),
  complex(.4,-.45)}
hyper.tikzBegin()
for _,P in ipairs(points) do
  hyper.drawPointOrbit(P, phi, nbIterations)
  hyper.drawHorocycle(limit, P, "red, draw
      ↪ opacity=.3")
end
hyper.tikzEnd()
\end{luacode*}
```



3.7 Triangle geometry (CAUTION)

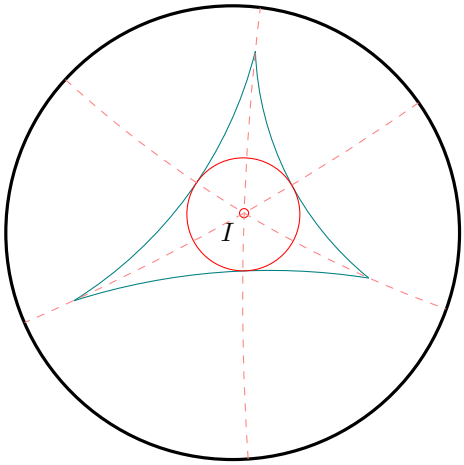
Centroid (intersection of medians)

```
\begin{luacode*}
hyper.tikzBegin()
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.3)
local C = complex(0.6,-0.2)
local AA = hyper.midpoint(B,C)
local BB = hyper.midpoint(C,A)
local CC = hyper.midpoint(A,B)
local G = hyper.triangleCentroid(A,B,C)
hyper.drawTriangle(A,B,C)
hyper.drawSegments(A, AA, B, BB, C, CC, "teal")
hyper.drawPoints(AA,BB,CC)
hyper.drawPoint(G,"white, draw=teal")
-- CAUTION :
local wrong = hyper.interpolate(A,AA,2/3)
hyper.drawPoint(wrong, "red")
hyper.tikzEnd()
\end{luacode*}
```



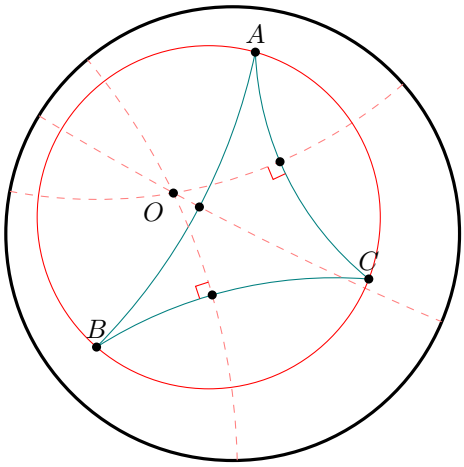
Incenter and incircle

```
\begin{luacode*}
hyper.tikzBegin()
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.3)
local C = complex(0.6,-0.2)
local I = hyper.triangleIncenter(A,B,C)
hyper.drawTriangle(A,B,C,"teal")
hyper.drawPoint(I,"white, draw=red")
hyper.labelPoint(I,"$I$", "below left")
hyper.drawIncircle(A,B,C,"red")
-- show intersection :
local mystyle = "dashed, red!50"
hyper.drawAngleBisector(A,B, C, mystyle)
hyper.drawAngleBisector(B,C,A, mystyle)
hyper.drawAngleBisector(C,A,B, mystyle)
hyper.tikzEnd()
\end{luacode*}
```



Circumcenter and circumcircle (CAUTION : does not always exist)

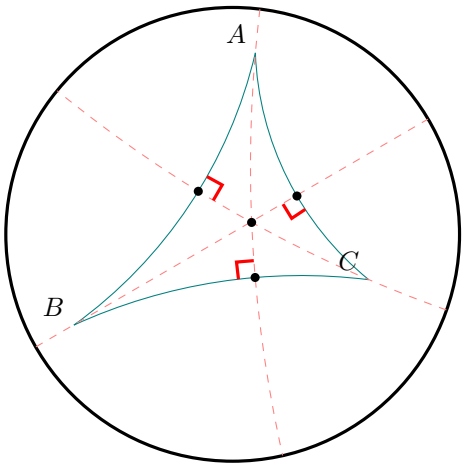
```
\begin{luacode*}
local A = complex(0.1,0.8)
local B = complex(-0.6,-0.5)
local C = complex(0.6,-0.2)
local O = hyper.triangleCircumcenter(A,B,C)
local BC = hyper.midpoint(B,C)
local CA = hyper.midpoint(C,A)
local AB = hyper.midpoint(A,B)
hyper.tikzBegin()
local mystyle = "dashed, red!50"
hyper.drawTriangle(A,B,C,"teal")
hyper.drawPerpendicularBisector(A, B, mystyle)
hyper.drawPerpendicularBisector(B, C, mystyle)
hyper.drawPerpendicularBisector(C, A, mystyle)
hyper.drawCircumcircle(A,B,C,"red")
hyper.labelPoint(O,"$O$", "below left")
hyper.drawRightAngle(O, CA, C,"red",1/8)
hyper.drawRightAngle(O,BC,B,"red",1/8)
hyper.drawPoints(O,A,B,C,AB,BC,CA)
hyper.labelPoints(A, "$A$", B, "$B$", C, "$C$",
    ↪ "above")
hyper.tikzEnd()
\end{luacode*}
```



Orthocenter (CAUTION : does not always exist)

```
\begin{luacode*}
local A = complex(0.1,0.8)
local B = complex(-0.7,-0.4)
local C = complex(0.6,-0.2)
local H = hyper.triangleOrthocenter(A,B,C)
local AA = hyper.projection(B,C)(A)
local BB = hyper.projection(C,A)(B)
local CC = hyper.projection(A,B)(C)

hyper.tikzBegin()
local mystyle = "dashed, red!50"
local style2 = "very thick, red"
hyper.drawTriangle(A,B,C,"teal")
hyper.drawPerpendicularThrough(A,B,C,mystyle)
hyper.drawPerpendicularThrough(B,C,A,mystyle)
hyper.drawPerpendicularThrough(C,A,B,mystyle)
hyper.drawRightAngle(A,AA,B,style2,1/15)
hyper.drawRightAngle(B,BB,C,style2,1/8)
hyper.drawRightAngle(C,CC,A,style2,1/12)
hyper.labelPoints(A, "$A$", B, "$B$", C, "$C$")
hyper.drawPoints(AA,BB,CC,H,"above")
hyper.tikzEnd()
\end{luacode*}
```



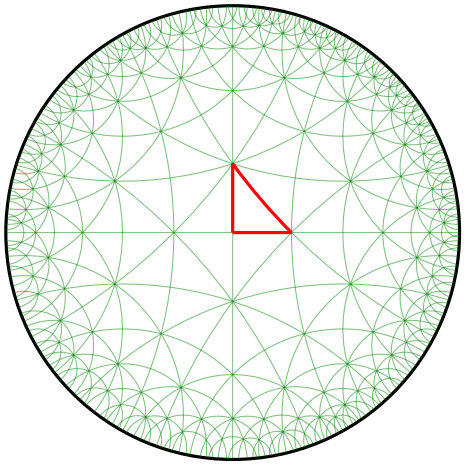
4 Tilings

The two following tilings are drawn with `TILING_DEPTH=12`

Triangle tiling

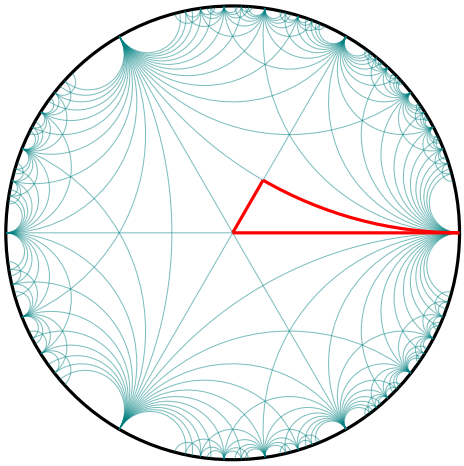
```
\begin{luacode*}
local A, B, C = hyper.fundamentalRightTriangle
    ↪ (4,5)
local geodesics = {
{hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}}
geodesics = hyper.propagateGeodesics(geodesics,
    ↪  TILING_DEPTH)

hyper.tikzBegin()
hyper.drawLinesFromTable(geodesics, "green!50!
    ↪ black, draw opacity=.5")
hyper.drawTriangle(A,B,C,"red, very thick")
hyper.tikzEnd()
\end{luacode*}
```



Triangle tiling with one angle zero

```
\begin{luacode*}
local A, B, C = hyper.fundamentalIdealTriangle
    ↪ (3,2)
local geodesics = {
  {hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}
}
geodesics = hyper.propagateGeodesics(geodesics,
    ↪  TILING_DEPTH)
hyper.tikzBegin()
hyper.drawLinesFromTable(geodesics, "teal, draw
    ↪  opacity=.5")
hyper.drawTriangle(A,B,C,"red, very thick")
hyper.tikzEnd()
\end{luacode*}
```

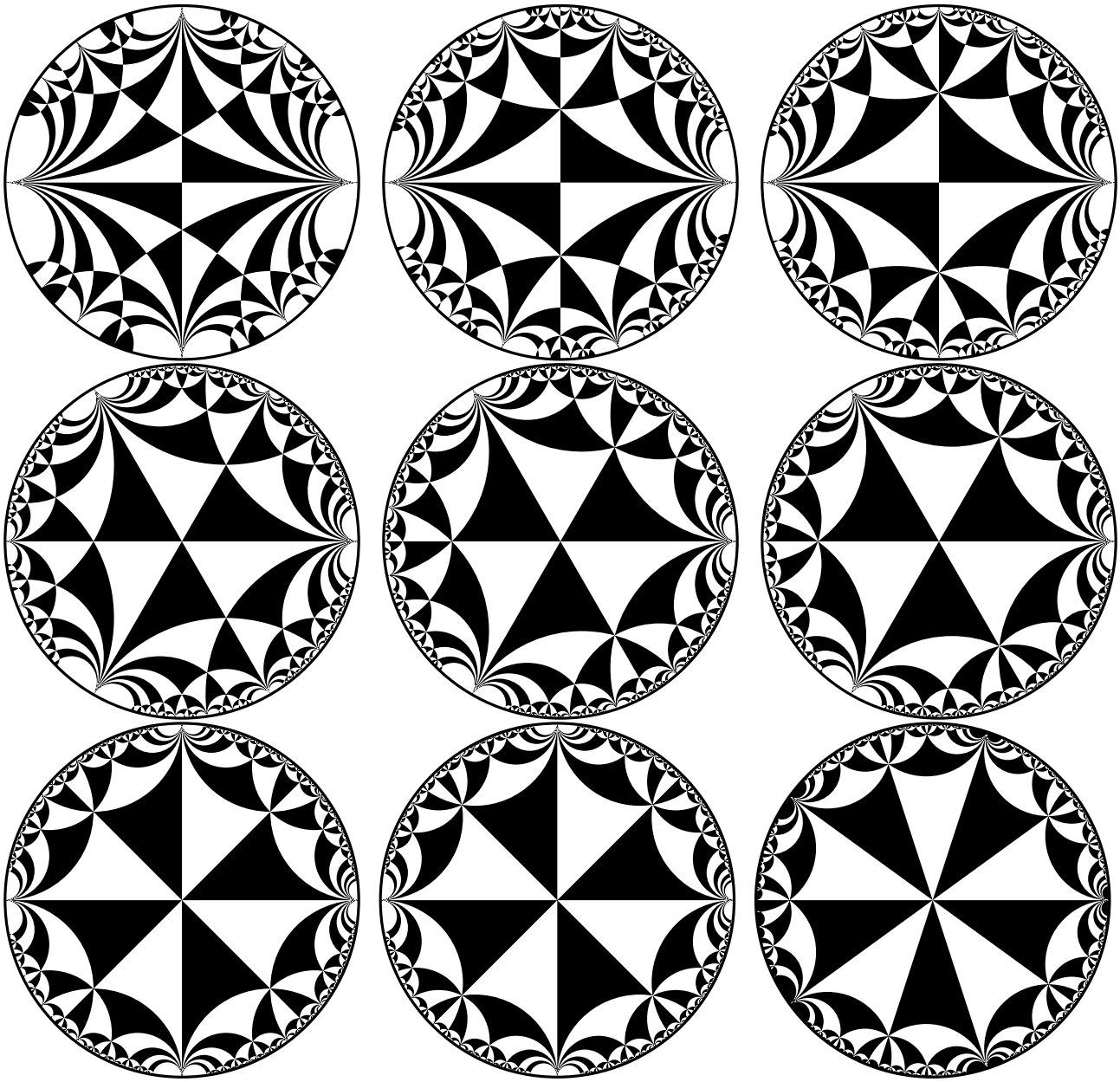


4.1 Source code of first picture

```
\begin{luacode*}
local A, B, C = hyper.fundamentalRightTriangle(4,5)
local phi = hyper.automorphism(C,0)
A,B,C = phi(A), phi(B), phi(C) -- center on pentagon
local geodesics = {
  {hyper.endpoints(A,B)},
  {hyper.endpoints(B,C)},
  {hyper.endpoints(C,A)}
}
geodesics = hyper.propagateGeodesics(geodesics, 24)
local shapes={}
for k,pair in ipairs(geodesics) do
  table.insert(shapes,hyper.shape_closed_line(pair[1], pair[2]))
end
shapes_string = table.concat(shapes, " ")
hyper.tikzBegin("scale=6")
hyper.tikzPrintf("\\fill[even odd rule] (0,0) circle (1) %s ;",shapes_string)
```

```
hyper.drawRegularPolygon(complex.ZERO, B,5,  "line width=.15cm, cyan")
hyper.tikzEnd()
\end{luacode*}
```

4.2 More images just for fun



5 Overview of the lua libraries

The Lua files are in the `lua/` subdirectory of the repo.

To build the `luahyperbolic.sty` from the lua source files, run `lua build.lua` on the root directory. (Lua must be installed)

The Lua source files are not needed to compile a latex document using luahyperbolic, only the sty file is necessary.

5.1 complex.lua

Functions

___add	___unm	det	isClosePair	isNot	rotate
___div	abs	distinct	isColinear	isReal	rotate90
___eq	abs2	dot	isComplex	isUnit	scal
___mul	arg	exp	isImag	log	toPolar
___pow	clone	exp_i	isInteger	new	unit
___sub	coerce	invert	isNear	nonzero	
___tostring	conj	isClose	isNearInteger	polar	

Constants (numbers and complex numbers)

EPS, EPS_INV, I, J, ONE, ZERO, ___index,

5.2 luahyperbolic-tikz (requires and copies luahyperbolic-core)

Functions beginning with underscore are available in the module but should not be needed for normal use. Don't use them, they could become private in the future and not be available to users anymore.

Functions

_assert	dist	drawRayFromPoints
_assert_in_closed_disk	distance	drawRayFromVector
_assert_in_disk	distance_to_geodesic	drawRegularPolygon
_circle_to_euclidean	drawAngle	drawRightAngle
_coerce_assert_in_closed_disk	drawAngleBisector	drawSegment
_coerce_assert_in_disk	drawArc	drawSegments
_distance_to_origin	drawCircle	drawSemicircle
_ensure_order	drawCircleRadius	drawTangentAt
_error	drawCircleThrough	drawTangentVector
_geodesic_data	drawCircumcircle	drawTriangle
_in_closed_disk	drawHorocycle	endpoints
_in_disk	drawHypercycleThrough	endpoints_angle_bisector
_in_half_plane	drawIncircle	endpoints_perpendicular_bisector
_metric_factor	drawLabeledPoint	expMap
_midpoint_to_origin	drawLine	fundamentalIdealTriangle
_on_circle	drawLineFromVector	fundamentalRightTriangle
_on_geodesic	drawLines	interCC
_radial_half	drawLinesFromTable	interLC
_radial_scale	drawPerpendicularBisector	interLL
_same_distance	drawPerpendicularThrough	interpolate
_warning	drawPoint	labelPoint
acosh	drawPointOrbit	labelPoints
angle	drawPoints	midpoint
asinh	drawPolySeg	mobiusTransformation
atanh	drawPolySegFromTable	parabolic
automorphism	drawPolygon	parabolic_fix1
automorphismFromPairs	drawPolygonFromTable	pointOrbit
automorphismSending	drawPolyline	projection
barycenter2	drawPolylineFromTable	propagateGeodesics
cosh	drawRay	randomPoint

reflection
rotation
rotationFromPair
shape_closed_line
shape_euclidean_segment
shape_segment
sinh
symmetry
symmetryAroundMidpoint
tangentVector
tanh
tikzBegin
tikzClearBuffer
tikzEnd
tikzExport
tikzGetFirstLines
tikzPrintf
tikz_define_nodes
triangleCentroid
triangleCircumcenter
triangleIncenter
triangleOrthocenter

Constants

Numbers : DRAW_EPS, DRAW_EPS_INV, DRAW_POINT_RADIUS, EPS, SCALE, tikzNbPicturesExported,

Strings : TIKZ_BEGIN_DISK, module, tikzOptions,

Tables : tikzBuffer,